Software Requirements Specification for Lake: subtitle describing software

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

October 18, 2026

Table 1: Revision History

Date	**Developer(s)**	**Change**
2026-10-09	Colin Chambachan	Switch from the Volere template to the Meyer template
Date2	Name(s)	Description of changes
...	...	...

Contents

Changes from Original SRS Meyer Template **1**

Glossary **3**

(G) Goals **4**
G.1 Context and Overall Objective . . . 4
G.2 Current Situation . . . 4
G.3 Expected Benefits . . . 4
G.4 Functionality Overview . . . 4
G.5 High-Level Usage Scenarios . . . 4
G.6 Limitations and Exclusions . . . 5
G.7 Stakeholders and Requirements Sources . . . 5
G.8 User Profiles . . . 5

(E) Environment **6**
E.1 Glossary . . . 6
E.2 Environment Components . . . 6
E.3 Constraints . . . 6
E.4 Assumptions . . . 6
E.5 Effects . . . 6
E.6 Invariants . . . 7
E.7 Standards and Legal/Regulatory Factors . . . 7
Environmental Requirements . . . 7

(S) System **8**
S.1 Components . . . 8
S.2 Functionality . . . 8
S.2.1 Component Name . . . 8
S.2.2 Non-Functional Requirements . . . 8
S.3 Interfaces . . . 8
S.4 Normal Operation . . . 9
S.5 Undesired Event Handling . . . 9
S.6 Detailed Usage Scenarios . . . 9
S.7 Prioritization . . . 10
S.8 Verification and Acceptance Criteria . . . 10
S.9 Formalizations . . . 10

S.10 Likely Changes to Requirements 11
S.11 Unlikely Changes to Requirements 11

(P) Project **13**
P.1 Roles and Personnel 13
P.2 Imposed Technical Choices 13
P.3 Schedule and Milestones 13
P.4 Tasks and Deliverables 13
P.5 Required Technology Elements 14
P.6 Risk and Mitigation Analysis 14
P.7 Requirements Process and Report 14

Safety and Security Requirements **15**

Requirement Dependencies **16**

1 Appendix — Generative AI Use Disclosure **18**
1.1 AI Tools Used 18
1.2 How and Where Used 18
1.3 Verification of AI-Generated Content 18

2 Appendix — Reflection **19**

List of Tables

1 Revision History i
2 Changes from Template 2
3 Environmental Requirements 7
4 Prioritization of Component Name requirements 10
5 Project Milestones and Due Dates 13
6 Tasks and Deliverables 14

List of Figures

Changes from Original SRS Meyer Template

[Meyer's template is a book chapter, not a course template, so every team adapts it. Record each structural change you make and why, so a reader who knows the original can follow this document. The rows below describe the adaptations already present in this file; add to them as the document evolves. —TPLT]

Table 2: Changes from Template

Change	**Reason**
Glossary (E.1) moved to the front of the document	Terms and acronyms must be defined before first use, and information should live in one place. E.1 now points here.
Requirement tables added to each PEGS chapter	Meyer's template describes requirements in prose. Labelled tables of functional (FR) and non-functional (NFR) requirements give each requirement an ID that can be prioritized, traced and verified.
Added G.8 User Profiles	Meyer lists stakeholders (G.7) but has no place for the distinct characteristics of each user group.
Added E.7 Standards and Legal/Regulatory Factors	Meyer's Environment chapter has no explicit home for standards, licensing or legal constraints.
Added S.4 Normal Operation and S.5 Undesired Event Handling	Separates expected behaviour from error handling instead of mixing both into S.2 Functionality.
Added S.9 Formalizations	Provides a place for mathematical or formal specification of the parts of the system that benefit from it.
Added Safety and Security Requirements	Lets the requirements from the Hazard Analysis be ported into this document.
Added Requirement Dependencies	Shows traceability between requirements visually.
Added References	Meyer's template has no references section.
Added Appendices (Generative AI Use Disclosure, Reflection)	Required by the course.
Added S.10 Likely changes to requirements	Required by the SRS rubric
Added S.11 Unlikely changes to requirements	Required by the SRS rubric

Glossary

[Define every domain term and acronym used in this document, in alphabetical order. Define a term once, here, and use it consistently everywhere else. Where a term has a good external definition, link to it and cite the source. —TPLT]

[Tip from the source template: define a `\hypertarget` for each glossary entry and a macro such as `\newcommand{\Term}{\hyperlink{glossary-term}{Term}}` so each use of the term in the body links back to its definition. —TPLT]

- **Term** – Definition.
- **Acronym (Expansion)** – Definition.

(G) Goals

[The Goals chapter is written for decision makers. It explains why the project exists, in the language of the business or domain, not of the software. It should be readable on its own. —TPLT]

G.1 Context and Overall Objective

[One or two paragraphs: the domain the system lives in, the problem it addresses, and the overall objective in a sentence a sponsor would recognize. —TPLT]

G.2 Current Situation

[How things are done today without the system, and what is wrong or missing. Reference existing tools or processes the system will replace or complement. —TPLT]

G.3 Expected Benefits

[The improvements the system will bring over the current situation, from the point of view of the stakeholders. Where possible, make each benefit measurable so it can be checked later. —TPLT]

- **Benefit** – Description.

G.4 Functionality Overview

[A high-level list of what the system does. Each item should correspond to one or more components in S.1 and requirements in S.2, but stay at the level a non-technical reader can follow. —TPLT]

- **Capability** – Description.

G.5 High-Level Usage Scenarios

[The main ways users will interact with the system, as short narratives (two or three sentences each). Detailed use cases belong in S.6. An activity diagram can be included here if it helps. —TPLT]

- **Scenario 1: Name** – Description.

G.6 Limitations and Exclusions

[What the system will not do. State exclusions explicitly so nobody later assumes they were simply forgotten. —TPLT]

- **Exclusion** – Description and rationale.

G.7 Stakeholders and Requirements Sources

[Stakeholders: every person or group affected by the system, including clients, users, supervisors, and the wider community. Requirements sources: the documents, people, standards, and existing systems the requirements were elicited from. —TPLT]

Stakeholders:

- **Stakeholder** – Their interest in the system.

Requirements Sources:

- **Source** – What it contributed.

G.8 User Profiles

[For each distinct group of hands-on users, describe their domain knowledge, goals, technical literacy, and which parts of the system matter most to them. These profiles justify usability and accessibility requirements later on. —TPLT]

1. Profile Name

- **Domain Knowledge:**
- **Goals:**
- **Technical Literacy:**
- **Rationale:** Which requirements exist for this user.

(E) Environment

[The Environment chapter describes the world the system operates in and cannot change: physical constraints, other systems, rules of the domain. These hold whether or not the system is built. —TPLT]

E.1 Glossary

The glossary has been moved to the front of the document (see the Glossary and the Changes from Original SRS Meyer Template).

E.2 Environment Components

[Elements of the environment that may affect or be affected by the system and the project, including other systems the system must interface with, hardware it runs on, and the people who operate it. —TPLT]

- **Component** – Description and its relationship to the system.

E.3 Constraints

[Obligations and limits imposed on the system by the environment: regulations, physical limits, resource limits, deadlines. These are not design choices; the system must live with them. —TPLT]

- **Constraint** – Description.

E.4 Assumptions

[Properties of the environment that the system relies on but cannot enforce. Each assumption is a risk if it turns out false; cross-reference the Hazard Analysis where relevant. —TPLT]

- **Assumption** – Description.

E.5 Effects

[The ways the system changes its environment once deployed: effects on users, on other systems, on resources. —TPLT]

- **Effect** – Description.

E.6 Invariants

[Properties of the environment that must remain true throughout the system's operation, and that the system must preserve. —TPLT]

- Invariant.

E.7 Standards and Legal/Regulatory Factors

[Coding, documentation, interface or safety standards the project will follow, and legal or regulatory factors such as licensing, intellectual property, and data privacy. Cite each standard. —TPLT]

Standards:

- **Standard** – What it governs and how it applies.

Legal and Regulatory Factors:

- **Factor** – Description.

Environmental Requirements

[Requirements that follow directly from this chapter: installability on the target platforms, resource limits, invariant enforcement, standards compliance, legal compliance. Label each one so it can be referenced from S.7 and S.8. —TPLT]

Table 3: Environmental Requirements

Label	Requirement
NFR.E.1	The system shall ...
NFR.E.2	The system shall ...

(S) System

[The System chapter is the technical core: what the system does, split by component, with each requirement labelled, prioritized and verifiable. —TPLT]

S.1 Components

[The major parts of the system and how they interact. Include a component diagram. The components named here are reused as the subsections of S.2 and the groupings of S.7. —TPLT]

- **Component** – Responsibility.

S.2 Functionality

[One subsubsection per component from S.1, each listing its functional requirements with labels FR.S.¡component¿.¡n¿. Each requirement is a single, testable "shall" statement. Non-functional requirements that cut across components go in the last subsubsection as NFR.S.¡n¿. —TPLT]

S.2.1 Component Name

Functional Requirements

- **FR.S.1.1** The system shall ...
- **FR.S.1.2** The system shall ...

S.2.2 Non-Functional Requirements

- **NFR.S.1 Quality attribute:** The system shall ...

S.3 Interfaces

[The interfaces the system exposes or consumes: user interfaces, APIs, protocols, file formats, hardware interfaces. For each, say who or what is on the other side. —TPLT]

- **Interface** – Description.

S.4 Normal Operation

[The expected flow of the system when everything goes right, from start-up to shut-down. Keep it to the main path; alternatives and failures go in S.5 and S.6. —TPLT]

1. Step.

S.5 Undesired Event Handling

[How the system responds to failures, invalid inputs, and unexpected conditions. Each entry should name the event and the required response. Cross-reference the Hazard Analysis. —TPLT]

- **Event** – Required response.

S.6 Detailed Usage Scenarios

[Full use cases for the scenarios introduced in G.5, one per scenario, in the structure below. The main success scenario should be numbered so extensions can reference its steps. —TPLT]

Use Case Name: Name
Primary Actor: Actor
Stakeholders and Interests:

- Stakeholder: interest.

Preconditions:

- Precondition.

Postconditions (Success Guarantees):

- Postcondition.

Postconditions (Failure Guarantees):

- Postcondition.

Main Success Scenario (Basic Flow):

1. Step.

Extensions:

- **Na.** Alternative or failure at step N, and how it is handled.

Justification: Why this scenario matters.

S.7 Prioritization

[Every requirement from S.2 and the Environmental Requirements table, with a MoSCoW priority and the reason for it. Group the tables by component. The Date column records when the priority was last decided. —TPLT]
Priorities follow MoSCoW: **M** must have, **S** should have, **C** could have, **W** won't have (this release).

Component Name

ID	Requirement Name	Priority	Reasoning	Date
FR.S.1.1	Name	M	Reasoning	YYYY-MM-DD
FR.S.1.2	Name	S	Reasoning	YYYY-MM-DD

Table 4: Prioritization of Component Name requirements

S.8 Verification and Acceptance Criteria

[How each class of requirement will be shown to be satisfied: unit and integration tests, measurements, user studies, inspections. Make the criteria concrete enough that the V&V Plan can be derived from them. Every requirement in S.2 should map to at least one criterion here. —TPLT]
Validation Overview:
Verification Approach by Requirement Class:

- **Class of requirements** – How it will be verified and what counts as acceptance.

S.9 Formalizations

[Mathematical or formal specification of the parts of the system that benefit from precision: state models, invariants, algorithms, data formats. Z notation is available through the `zed-csp` package. If nothing in the system warrants formalization, say so and why. —TPLT]

S.10 Likely Changes to Requirements

- **LC.1** A Lake cluster cannot recover from a failing leader node (see FR.S.1.25). In the future, the ability to recover from a failing leader node may be added.

- **LC.2** Lake only supports desktop operating systems (Windows, macOS, Linux) (see NFR.S.13). Support for mobile devices (iOS, Android) may be added once other goals and requirements have been met.

- **LC.3** Lake assumes the local area network being used to connect the devices is private and trusted (See Hazard Analysis Section 4 - Critical Assumptions). Lake's may be extended with additional security hardening to allow clusters to run on public or untrusted networks.

- **LC.4** Lake requires that each device manually confirm joining a cluster (see FR.S.1.17 and FR.S.1.21). In the future, a trust mechanism may be introduced, which would allow devices to automatically join clusters created by trusted devices without requiring confirmation.

- **LC.5** Requirement thresholds and intervals will likely change, as real-world usage gives better estimates for reasonable threshold values.

- **LC.6** It is likely that future versions of Lake will support a headless mode where all UI interactions are performed through a command line interface. This would be useful for users wishing to use headless servers as nodes.

S.11 Unlikely Changes to Requirements

- **UC.1** Model training and fine-tuning support is unlikely to be added to Lake. Lake is primarily an inference framework, so the addition of training would be out of scope for such a project.

- **UC.2** Additional input model formats aside from GGUF (TODO: glossary) are unlikely to be added, as GGUF is one of the most common standards for LLMs (TODO: glossary). Additionally, models in other formats can be converted to GGUF so there is little reason for Lake to support them directly.

- **UC.3** A primary goal of Lake is privacy (see NFR.S.11), so modifications that would compromise privacy like incorporating cloud-based services are unlikely.
- **UC.4** Even though one of Lake's goals is to run larger models, integrating quantization tools into Lake is unlikely. Lake's scope is focused on distributed inference, and quantization tools would better fit in the scope of an independent project.

(P) Project

[The Project chapter covers how the system will be built: people, imposed choices, schedule, deliverables, risks, and how requirements will be managed over the project's life. —TPLT]

P.1 Roles and Personnel

[Each team member and their role, plus the supervisor and TA. Keep this consistent with the Development Plan. —TPLT]

- **Role (Name)** – Responsibilities.

P.2 Imposed Technical Choices

[Technologies that are fixed before design begins and why: languages, frameworks, hardware, platforms mandated by the client, supervisor, or environment. Choices the team is still free to make do not belong here. —TPLT]

- **Choice** – What is imposed and by whom.

P.3 Schedule and Milestones

[The major milestones and their dates. Course deliverables are a starting point; add project-specific milestones such as prototypes or integration points. —TPLT]

Milestone	Due Date
Milestone	Date

Table 5: Project Milestones and Due Dates

P.4 Tasks and Deliverables

[The work breakdown: each task, what it produces, and which milestone it feeds. Tasks should be small enough to assign to one or two people. —TPLT]

Task	Description / Steps	Deliverable	Due Date
Task	Description	Deliverable	Date

Table 6: Tasks and Deliverables

P.5 Required Technology Elements

[Everything the project needs to obtain or build before the system can be delivered: libraries, hardware, datasets, services, licences, compute. Note anything not yet secured. —TPLT]

- **Element** – Purpose and status.

P.6 Risk and Mitigation Analysis

[Project risks (schedule, technical, people, external dependencies), each with its likelihood, impact, and mitigation. System hazards belong in the Hazard Analysis, not here. —TPLT]

- **Risk: Name**
  - Description:
  - Mitigation:

P.7 Requirements Process and Report

[How requirements were elicited and how they will be maintained: who was consulted, through which channels, how changes are tracked, and how this document will be kept in sync with the Hazard Analysis, V&V Plan and design. —TPLT]

- **Elicitation** –
- **Communication** –
- **Change Tracking** –
- **Process Updates** –

Safety and Security Requirements

[Port the requirements from the Safety and Security Requirements section of the Hazard Analysis so this document is the single list of requirements. Keep the labels identical in both documents. —TPLT]

Functional Safety Requirements

- **FR.Sa.1** The system shall ...

Non-Functional Safety Requirements

- **NFR.Sa.1** The system shall ...

Requirement Dependencies

[A traceability diagram or matrix showing which requirements depend on which. Use the landscape environment below for a wide figure. —TPLT]

[Cite sources with \citep{key} or \citet{key}; entries live in refs/References.bib. The References section appears once at least one source is cited. —TPLT]

1 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

1.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

1.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

1.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

2 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?